



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

4

Parallelism and Concurrent Programming

Computing performance—typically measured in millions of instructions per second (MIPS) or floating-point operations per second (FLOPS)—has been improving at a staggeringly rapid and consistent rate over the past four decades. In the late 1970s, the Intel 8087 floating-point coprocessor could muster only about 50 kFLOPS (5×10^4 FLOPS), while at roughly the same time a Cray-1 supercomputer the size of a large refrigerator could operate at a rate of roughly 160 MFLOPS (1.6×10^8 FLOPS). Today, the CPU in game consoles like the Playstation 4 or the Xbox One produces roughly 100 GFLOPS (10^{11} FLOPS) of processing power, and the fastest supercomputer, currently China's Sunway TaihuLight, has a LINPACK benchmark score of 93 PFLOPS (peta-FLOPS, or a staggering 9.3×10^{16} floating-point operations per second). This is an improvement of seven orders of magnitude for personal computers, and eight orders of magnitude for supercomputers.

Many factors contributed to this rapid rate of improvement. In the early days, the move from vacuum tubes to solid-state transistors permitted the miniaturization of computing hardware. The number of transistors that could be etched onto a single chip rose dramatically as new kinds of transistors, new types of digital logic, new substrate materials and new manufacturing processes were developed. These advances also contributed to improvements in

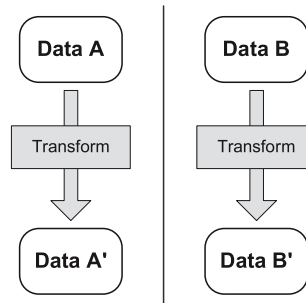


Figure 4.1. Two flows of control operating on *independent* data are not considered concurrent because they are not prone to data races.

power consumption and dramatic increases in CPU clock speeds. And starting in the 1990s, computer hardware manufacturers have increasingly turned to *parallelism* as a means of improving computing performance.

Writing software that runs *correctly* and *efficiently* on parallel computing hardware is significantly more difficult than writing software for the serial computers of yesteryear. It requires a deep understanding of how the hardware actually works. What's more, taking full advantage of the multicore CPUs found in modern computing platforms requires an approach to software design called *concurrent programming*. In a concurrent software system, multiple flows of control cooperate to solve a common problem. These flows of control must be carefully coordinated. Many of the techniques that work well in serial programs break down when applied to concurrent programs. As such, it's important for modern programmers (in all industries, including games) to have a solid understanding of parallel computing hardware, and to be well versed in concurrent programming techniques.

4.1 Defining Concurrency and Parallelism

4.1.1 Concurrency

A concurrent piece of software utilizes *multiple flows of control* to solve a problem. These flows of control might be implemented as multiple *threads* running within the context of a single process, or multiple cooperating processes running either on a single computer or multiple computers. Multiple flows of control can also be implemented within a process using other techniques such as *fibers* or *coroutines*.

The primary factor that distinguishes *concurrent programming* from *sequential programming* is the reading and/or writing of *shared data*. As shown in

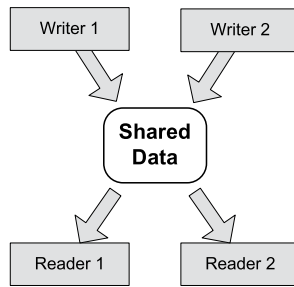


Figure 4.2. Two flows of control both reading from a shared data file and/or writing to a shared data file are examples of *concurrency*.

Figure 4.1, if we have two or more flows of control, each operating on a totally independent block of data, then this is not technically an example of concurrency—it’s just “computing at the same time.”

The central problem of concurrent programming is how to coordinate multiple readers and/or multiple writers of a shared data file, in such a way as to ensure predictable, correct results. At the heart of this problem is a special kind of *race condition* known as a *data race*, in which two or more flows of control “compete” to be the first to read, modify and write a chunk of shared data. The crux of the concurrency problem is to identify and eliminate data races. Two examples of concurrency are illustrated in Figure 4.2.

We’ll explore the techniques programmers use to avoid data races and thereby write reliable concurrent programs starting in Section 4.5. But before we do that, let’s take a look at how parallel computer hardware can both provide an effective platform for running concurrent software, and improve the execution speed of sequential programs as well.

4.1.2 Parallelism

In computer engineering, the term *parallelism* refers to any situation in which two or more distinct hardware components are operating simultaneously. In other words, *parallel* computer hardware can perform more than one task at a time. In contrast, *serial* computer hardware is capable of doing only one thing at a time.

Prior to 1989, consumer-grade computing devices were exclusively serial machines. Examples include the MOS Technology 6502 CPU, which was used in the Apple II and Commodore 64 personal computers, and the Intel 8086, 80286 and 80386 CPUs which were at the heart of early IBM PCs and clones.

Today, parallel computing hardware is ubiquitous. One obvious example of hardware parallelism is a multicore CPU, such as the Intel Core™ i7 or the AMD Ryzen™ 7. But parallelism can be employed at a wide range of granularities. For example, a single CPU might contain multiple ALUs and therefore be capable of performing multiple independent calculations in parallel. And at the other end of the spectrum, a cluster of computers working in tandem to solve a common problem is also an example of hardware parallelism.

4.1.2.1 Implicit versus Explicit Parallelism

One way to classify the various forms of parallelism in computer hardware design is to consider the *purpose* of parallelism in each. In other words, what problem does parallelism solve in a given design? Thinking along these lines, we can divide parallelism coarsely into two categories:

- implicit parallelism, and
- explicit parallelism.

Implicit parallelism refers to the use of parallel hardware components within a CPU for the purpose of improving the performance of a *single* instruction stream. This is also known as *instruction level parallelism* (ILP), because the CPU executes instructions from a single stream (a single thread) but each instruction is executed with some degree of hardware parallelism. Examples of implicit parallelism include:

- pipelining,
- superscalar architectures, and
- very long instruction word (VLIW) architectures.

We'll explore implicit parallelism in Section 4.2. GPUs also make extensive use of implicit parallelism; we'll take a closer look at the design and programming of GPUs in Section 4.11.

Explicit parallelism refers to the use of duplicated hardware components within a CPU, computer or computer system, for the purpose of running *more than one instruction stream* simultaneously. In other words, explicitly parallel hardware is designed to run *concurrent* software more efficiently than would be possible on a serial computing platform. The most common examples of explicit parallelism are:

- hyperthreaded CPUs,
- multicore CPUs,

- multiprocessor computers,
- computer clusters,
- grid computing, and
- cloud computing.

We'll investigate these explicitly parallel architectures further in Section 4.3.

4.1.3 Task versus Data Parallelism

Another way to understand parallelism is to divide it into two broad categories based on the kind of work being done in parallel.

- *Task parallelism*. When multiple heterogeneous operations are performed in parallel, we call this task parallelism. Performing animation calculations on one core while performing collision checks on another would be an example of this form of parallelism.
- *Data parallelism*. When a *single* operation is performed on multiple data items in parallel, it is called data parallelism. Calculating 1000 skinning matrices by running 250 matrix calculations on each of four cores would be an example of data parallelism.

Most real concurrent programs make use of both task and data parallelism to varying degrees.

4.1.4 Flynn's Taxonomy

Yet another way to classify the varying degrees of parallelism we'll encounter in computing hardware is to use *Flynn's Taxonomy*. Proposed by Michael J. Flynn of Stanford University in 1966, this approach breaks down parallelism into a two-dimensional space. Along one axis, we have the number of parallel flows of control (which Flynn refers to as the number of instructions running in parallel at any given moment). On the other axis, we have the number of distinct data streams being operated on by each instruction in the program. The space is thus divided into four quadrants:

- *Single instruction, single data (SISD)*: A single instruction stream operating on a single data stream.
- *Multiple instruction, multiple data (MIMD)*: Multiple instruction streams operating on multiple independent data streams.
- *Single instruction, multiple data (SIMD)*: A single instruction stream operating on multiple data streams (i.e., performing the same sequence of operations on multiple independent streams of data simultaneously).

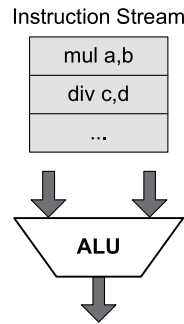


Figure 4.3. Example of SISD. A single ALU performs the multiply first, followed by the divide.

- *Multiple instruction, single data (MISD)*: Multiple instruction streams all operating on a single data stream. (MISD is rarely used in games, but one common application is to provide fault tolerance via redundancy.)

4.1.4.1 Single versus Multiple Data

It's important to realize here that a "data stream" isn't just an array of numbers. Most arithmetic operators are binary—they operate on two inputs to produce a single output. When applied to binary arithmetic, the term "single data" refers to a single *pair* of inputs, with a single output. As an example, let's have a look at how two binary arithmetic operations, a multiply ($a \times b$) and a divide (c/d), might be accomplished under each of the four Flynn categories:

- In a SISD architecture, a single ALU performs the multiply first, followed by the divide. This is illustrated in Figure 4.3.
- In a MIMD architecture, two ALUs perform operations in parallel, operating on two independent instruction streams. This is shown in Figure 4.4.
- The MIMD classification also applies to the case in which a single ALU processes two independent instruction streams via *time-slicing*, as shown in Figure 4.5.
- In a SIMD architecture, a single "wide ALU" known as a vector processing unit (VPU) performs the multiply first, followed by the divide, but each instruction operates on a pair of four-element input vectors and produces a four-element output vector. Figure 4.6 illustrates this approach.

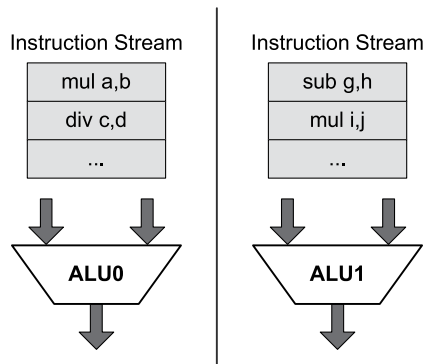


Figure 4.4. Example of MIMD. Two ALUs perform operations in parallel.

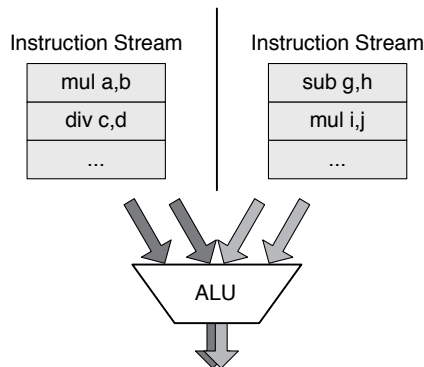


Figure 4.5. Example of time-sliced MIMD. A single ALU performs operations on behalf of two independent instruction streams, perhaps by alternating between them.

- In a MISD architecture, two ALUs process the same instruction stream (multiply first, followed by divide) and ideally produce identical results. Illustrated in Figure 4.7, this architecture is primarily useful for implementing fault tolerance via redundancy. ALU 1 acts as a “hot spare” for ALU 0 and vice-versa, meaning that if one of the ALUs experiences a failure, the system can seamlessly switch to the other.

4.1.4.2 GPU Parallelism: SIMT

In recent years, a fifth classification has been added to Flynn’s taxonomy to account for the design of graphics processing units (GPU). *Single instruction multiple thread* (SIMT) is essentially a hybrid between SIMD and MIMD, used primarily in the design of GPUs. It mixes SIMD processing (a single instruc-

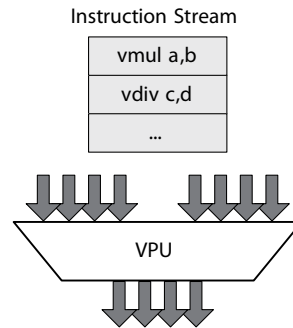


Figure 4.6. Example of SIMD. A single vector processing unit (VPU) performs the multiply first, followed by the divide, but each instruction operates on a pair of four-element input vectors and produces a four-element output vector.

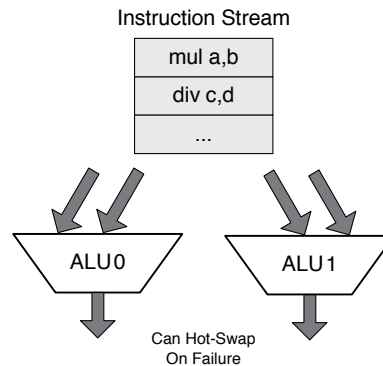


Figure 4.7. Example of MISD. Two ALUs process the same instruction stream (multiply first, followed by divide) and ideally produce identical results.

tion operating on multiple data streams simultaneously) with multithreading (more than one instruction stream sharing a processor via time-slicing).

The term “SIMT” was coined by NVIDIA, but it can be used to describe the design of any GPU. The term *manycore* is also often used to refer to a SIMT design (i.e., a GPU consisting of a relatively large number of lightweight SIMD cores) whereas the term *multicore* refers to MIMD designs (i.e., a CPU with a relatively smaller number of heavyweight general purpose cores). We’ll take a closer look at the SIMT design employed by GPUs in Section 4.11.

4.1.5 Orthogonality of Concurrency and Parallelism

We should stress here that concurrent software doesn't *require* parallel hardware, and parallel hardware isn't *only* for running concurrent software. For example, a concurrent multithreaded program can run on a single, serial CPU core via preemptive multitasking (see Section 4.4.4). Likewise, instruction level parallelism is intended to improve the performance of a single thread, and therefore benefits both concurrent and sequential software. So while they are closely related, concurrency and parallelism are really orthogonal concepts.

As long as our system involves *multiple readers* and/or *multiple writers* of a shared data object, we have a concurrent system. Concurrency can be achieved via preemptive multitasking (on serial *or* parallel hardware) or via true parallelism (in which each thread executes on a distinct core)—the techniques we'll learn in this chapter will be applicable either way.

4.1.6 Roadmap of the Chapter

In the following sections, we'll first turn our attention to implicit parallelism, and how best to optimize our software to take advantage of it. Next, we'll review the most common forms of explicit parallelism. Then we'll explore the various concurrent programming techniques used to harness explicitly parallel computing platforms. Finally, we'll round out the discussion of parallel programming by discussing SIMD vector processing, and how it applies to GPU design and general-purpose GPU programming (GPGPU) techniques.

4.2 Implicit Parallelism

In Section 4.1.2.1 we said that *implicit parallelism* is the use of parallel computing hardware for the purpose of improving the execution speed of a single thread. CPU manufacturers began using implicit parallelism in their consumer products in the late 1980s, in an attempt to make *existing code* run faster on each new generation of CPUs within a given product line.

There are a number of ways to apply parallelism to the problem of improving a CPU's single-threaded performance. The most common are *pipelined* CPU architectures, *superscalar* designs, and *very long instruction word* (VLIW) architectures. We'll begin by having a look at how a pipelined CPU functions, and then address the other two variants of implicit parallelism.

Clock Cycle	Fetch	Dec	Exec	Mem	WB
0	A				
1		A			
2			A		
3				A	
4					A
5	B				
6		B			

Figure 4.8. In a non-pipelined CPU, instruction stages are idle much of the time.

4.2.1 Pipelining

In order for a single machine language instruction to be executed by the CPU, it must pass through a number of distinct *stages*. Every CPU's design is a bit different—some CPU designs employ a large number of granular stages, while others utilize a smaller number of coarse-grained stages. However every CPU implements the following basic stages in one way or another:

- *Fetch*. The instruction to be executed is read from the memory location pointed to by the instruction pointer register (IP).
- *Decode*. The instruction word is decomposed into its opcode, addressing mode and optional operands.
- *Execute*. Based on the opcode, the appropriate functional unit within the CPU is selected (ALU, FPU, memory controller, etc.). The instruction is dispatched to the selected component for processing, along with any relevant operand data. The functional unit then performs its operation.
- *Memory access*. If the instruction involves reading or writing memory, the memory controller performs the appropriate operation during this stage.
- *Register write-back*. The functional unit executing the instruction (ALU, FPU, etc.) writes its results back into the destination register.

Figure 4.8 traces the path of two instructions named “A” and “B” through the five execution phases of a serial CPU. You’ll notice right away that this diagram contains a lot of blank space: While one stage is busy doing its thing to an instruction, all the other stages are twiddling their thumbs, doing nothing.

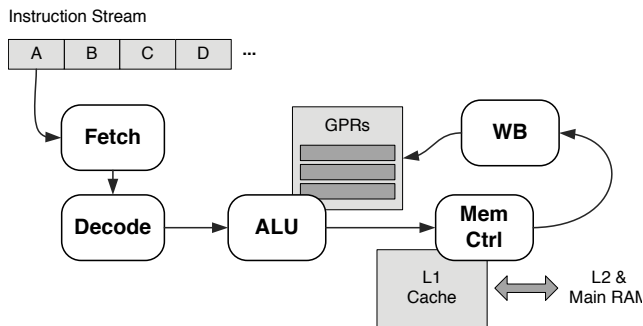


Figure 4.9. Components of a pipelined scalar CPU.

Clock Cycle	Fetch	Dec	Exec	Mem	WB
0	A				
1	B	A			
2	C	B	A		
3	D	C	B	A	
4	E	D	C	B	A
5	F	E	D	C	B
6	G	F	E	D	C

Figure 4.10. Ideal flow of instructions through a pipelined CPU.

Each of the stages of instruction execution is actually handled by different hardware within the CPU, as shown in Figure 4.9. The control unit (CU) and memory controller handle the instruction fetch stage. A different circuit within the CU then handles the decode stage. The ALU, FPU or VPU handles the lion's share of the execute stage. The memory stage is performed by the memory controller. And finally, the write-back stage primarily involves the registers. This division of labor amongst different circuits within the CPU is the key to making the CPU more efficient: We just need to keep all the stages' hardware busy all the time.

The solution is known as *pipelining*. Instead of waiting for each instruction to complete all five stages before starting to execute the next one, we begin the execution of a new instruction on *every* clock cycle. Multiple instructions are therefore "in flight" simultaneously. This process is illustrated in Figure 4.10.

Pipelining is a bit like doing laundry. If you have a large number of loads to do, it wouldn't be very efficient to wait until each load has been washed and

dried before starting the next one—while the washer is busy, the dryer would be sitting idle, and vice versa. It’s much better to keep both machines busy at all times, by starting the second load washing just as soon as the first load goes into the dryer, and so on.

Pipelining is a form of parallelism known as *instruction-level parallelism* (ILP). For the most part, ILP is designed to be transparent to the programmer. Ideally, at a given clock speed, a program that runs properly on a scalar CPU should be able to run correctly—but faster—on a pipelined CPU, as long as the two processors support the same instruction set architecture (ISA) of course. In theory, a CPU with a pipeline that is N stages deep can execute a program N times faster than its serial counterpart. However, as we’ll explore in Section 4.2.4, pipelining doesn’t always perform as well as we would expect, thanks to various kinds of *dependencies* between instructions in the instruction stream. Programmers who are interested in writing high-performance code therefore cannot remain oblivious to ILP. We must embrace it, understand it, and sometimes adjust the design of our code and/or data in order to get the most out of a pipelined CPU.

4.2.2 Latency versus Throughput

The *latency* of a pipeline is the amount of time required to completely process a single instruction. This is just the sum of the latencies of all the stages in the pipeline. Denoting latencies with the time variable T , we can write:

$$T_{\text{pipeline}} = \sum_{i=0}^{N-1} T_i \quad (4.1)$$

for a pipeline with N stages.

The *throughput* or *bandwidth* of a pipeline is a measure of how many instructions it can process per unit time. The throughput of a pipeline is determined by the latency of its slowest stage—much as a chain is only as strong as its weakest link. The throughput can be thought of as a frequency f , measured in instructions per second. It can be written as follows:

$$f = \frac{1}{\max(T_i)}. \quad (4.2)$$

4.2.3 Pipeline Depths

We said that each stage in a CPU can potentially have a different latency (T_i), and that the stage with the longest latency dictates the throughput of the entire processor. On each clock cycle, the other stages sit idle waiting for the longest

stage to complete. Ideally, then, we'd like all of the stages in our CPU to have roughly the same latency.

This goal can be achieved by increasing the total number of stages in the pipeline: If one stage is taking much longer than the others, it can be broken into two or more shorter stages in an attempt to make all the latencies roughly equal. However, we can't just keep subdividing stages forever. The larger the number of stages, the higher the overall instruction latency will be. This increases the cost of pipeline stalls (see Section 4.2.4). Therefore CPU manufacturers try to strike a balance between increasing throughput via deeper pipelines, and keeping the overall instruction latency in check. As a result, real CPU pipelines range from 4 or 5 stages at minimum, to something on the order of 30 stages at most.

4.2.4 Stalls

Sometimes the CPU is unable to issue a new instruction on a particular clock cycles. This is called a *stall*. On such a clock cycle, the first stage in the pipeline lies idle. On the next clock cycle, the second stage will be idle, and so on. A stall can therefore be thought of as a "bubble" of idle time that propagates through the pipeline at a rate of one stage per clock cycle. These bubbles are sometimes called *delay slots*.

4.2.5 Data Dependencies

Stalls are caused by *dependencies* between instructions in the instruction stream being executed. For example, consider the following sequence of instructions:

```
mov  ebx,5      ;; load the value 5 into register EBX
imul eax,10     ;; multiply the contents of EAX by 10
                ;; (result stored in EAX)
add  eax,7      ;; add 7 to EAX (result stored in EAX)
```

Ideally, we'd like to issue the `mov`, `imul` and `add` instructions on three consecutive clock cycles, to keep the pipeline as busy as possible. But in this case, the results of the `imul` instruction are used by the `add` instruction that follows it, so the CPU must wait until the `imul` has made it all the way through the pipeline before issuing the `add`. If the pipeline contains five stages, that means four cycles are wasted (see Figure 4.11). These kinds of dependencies between instructions are called *data dependencies*.

There are actually three kinds of dependencies between instructions that can cause stalls:

- data dependencies,

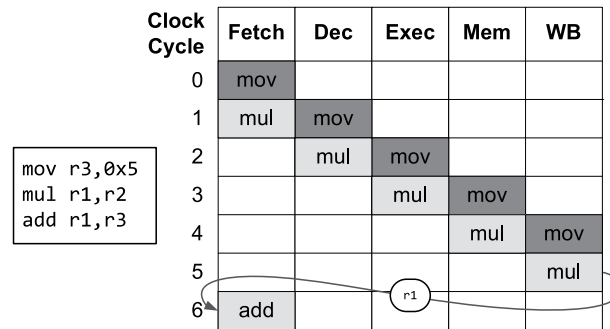


Figure 4.11. A data dependency between instructions causes a pipeline stall.

- control dependencies (also known as branch dependencies), and
- structural dependencies (also known as resource dependencies).

First we'll discuss how to avoid data dependencies, then we'll have a look at branch dependencies and how to mitigate their effects. Finally, we'll introduce superscalar CPU architectures and discuss how they can give rise to structural dependencies in a pipelined CPU.

4.2.5.1 Instruction Reordering

To mitigate the effects of a data dependency, we need to find some other instructions for the CPU to execute while it waits for the dependent instruction to make its way through the pipeline. This can often be accomplished by *re-ordering* the instructions in the program (while taking care not to change the *behavior* of the program in the process). For any given pair of interdependent instructions, we want to find some nearby instructions that are *not* dependent on them, and move those instructions up or down so that they end up running *between* the dependent instruction pair, thus filling the "bubble" with useful work.

Instruction reordering may of course be done by hand, by an adventurous programmer who doesn't mind diving into assembly language programming. Thankfully, however, this is often not necessary: Today's optimizing compilers are very good at reordering instructions automatically to reduce or eliminate the impact of data dependencies.

As programmers, of course we shouldn't blindly trust the compiler to optimize our code perfectly—when writing high-performance code, it's always a good idea to have a look at the disassembly and verify that the compiler did a reasonable job. But that said, we should also remember the 80/20 rule

(Section 2.3) and only spend time optimizing the 20% or less of our code that actually has a noticable impact on overall performance.

4.2.5.2 Out-of-Order Execution

Compilers and programmers aren't the only ones capable of reordering a sequence of machine language instructions to prevent stalls. Many of today's CPUs support a feature known as *out-of-order execution*, which enables them to dynamically detect data dependencies between instructions, and automatically resolve them.

To accomplish this feat, the CPU looks ahead in the instruction stream and analyzes the instructions' register usage in order to detect dependencies between them. When a dependency is found, the "look-ahead window" is searched for another instruction that is *not* dependent on any of the currently-executing instructions. If one is found, it is issued (out of order!) to keep the pipeline busy. The details of how out-of-order execution works is beyond our scope here. Suffice it to say that as programmers, we cannot rely on the CPU to execute the instructions in the same order we (or the compiler) wrote them.

Both the compiler's optimizer and the CPU's out-of-order execution logic take great care to ensure that the behavior of the program doesn't change as a result of instruction reordering. However, as we'll see in Section 4.9.3, compiler optimizations and out-of-order execution *can* cause bugs in a concurrent program (i.e., a program consisting of multiple threads that share data). This is one of the many reasons why concurrent programming requires more care than serial programming.

4.2.6 Branch Dependencies

What happens when a pipelined CPU encounters a conditional branch instruction (e.g., an `if` statement, or the conditional expression at the end of a `for` or `while` loop)? To answer this question, let's consider the following C/C++ code:

```
int SafeIntegerDivide(int a, int b, int defaultVal)
{
    return (b != 0) ? a / b : defaultVal;
}
```

If we looked at the disassembly for this function, it might look something like this on an Intel x86 CPU:

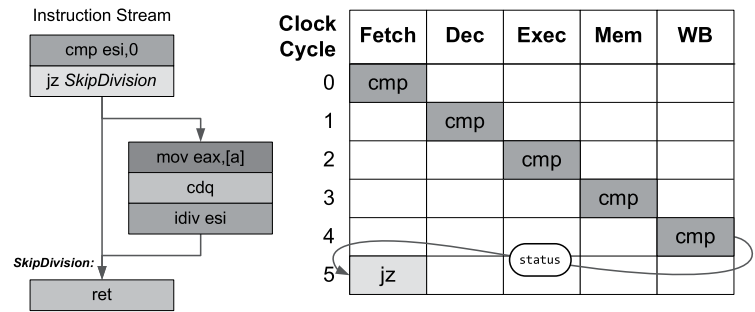


Figure 4.12. A dependency between a comparison instruction and a conditional branch instruction is called a branch dependency.

```
; function preamble omitted for clarity...

; first, put the default into the return register
mov  eax,dword ptr [defaultVal]

mov  esi,dword ptr [b] ; check (b != 0)
cmp  esi,0
jz   SkipDivision

mov  eax, dword ptr[a] ; divisor (a) must be in EDX:EAX
cdq                               ; ... so sign-extend into EDX
idiv esi                        ; quotient lands in EAX

SkipDivision:
; function postamble omitted for clarity...
ret                               ; EAX is the return value
```

The dependency here is between the `cmp` (compare) instruction and the `jz` (jump if equal to zero) instruction. The CPU cannot issue the conditional jump until it knows the results of the comparison. This is called a *branch dependency* (also known as a *control dependency*). See Figure 4.12 for an illustration of a branch dependency.

4.2.6.1 Speculative Execution

One way CPUs deal with branch dependencies is via a technique known as *speculative execution*, also known as *branch prediction*. Whenever a branch instruction is encountered, the CPU tries to *guess* at which branch is going to be taken. It continues to issue the instructions from the selected branch, in the hopes that its guess was correct. Of course, the CPU won't know for sure whether it guessed correctly until the dependent instruction pops out at the end of the pipeline. If the guess ends up being wrong, the CPU has executed

instructions that shouldn't have been executed at all. So the pipeline must be *flushed* and restarted at the first instruction of the correct branch. This is called a *branch penalty*.

The simplest guess a CPU can make is to assume that branches are never taken. The CPU just keeps executing instructions in sequential order, and only jumps the instruction pointer to a new location when its guess is proven wrong. This approach is I-cache friendly, in the sense that the CPU always prefers the branch whose instructions are most likely to be in the cache.

Another slightly more advanced approach to branch prediction is to assume that backward branches are always taken and forward branches are never taken. A backward branch is the kind found at the end of a `while` or `for` loop, so such branches tend to be more prevalent than forward branches.

Most high-quality CPUs include *branch prediction* hardware that can improve the quality these “static” guesses significantly. A branch predictor can track the results of a branch instruction over multiple iterations of a loop and discover patterns that help it make better guesses on subsequent iterations.

PS3 game programmers had to deal with the poor performance of “branchy” code all the time, because the branch predictors on the Cell processor were frankly pretty terrible. But the AMD Jaguar CPU found in the PS4 and Xbox One has highly advanced branch prediction hardware, so game programmers can breathe a little easier when writing code for the PS4.

4.2.6.2 Predication

Another way to mitigate the effects of branch dependencies is to simply avoid branching altogether. Consider again the `SafeIntegerDivide()` function, although we'll modify it slightly to work in terms of floating-point values instead of integers:

```
float SafeFloatDivide(float a, float b, float d)
{
    return (b != 0.0f) ? a / b : d;
}
```

This simple function calculates one of two answers, depending on the results of the conditional test `b != 0`. Instead of using a conditional branch statement to return one of these two answers, we can instead arrange for our conditional test to generate a *bit mask* consisting of all zeros (0x0U) if the condition is false, and all ones (0xFFFFFFFFU) if it is true. We can then execute *both* branches, generating two alternate answers. Finally, we use the mask to produce the final answer that we'll return from the function.

The following pseudocode illustrates the idea behind predication. (Note that this code won't run as-is. In particular, you can't mask a float with an unsigned int and obtain a float result—you'd need to use a union to reinterpret the bit patterns of the floats as if they were unsigned integers when applying the mask.)

```
int SafeFloatDivide_pred(float a, float b, float d)
{
    // convert Boolean (b != 0.0f) into either 1U or 0U
    const unsigned condition = (unsigned)(b != 0.0f);

    // convert 1U -> 0xFFFFFFFFU
    // convert 0U -> 0x00000000U
    const unsigned mask = 0U - condition;

    // calculate quotient (will be QNaN if b == 0.0f)
    const float q = a / b;

    // select quotient when mask is all ones, or default
    // value d when mask is all zeros (NOTE: this won't
    // work as written -- you'd need to use a union to
    // interpret the floats as unsigned for masking)
    const float result = (q & mask) | (d & ~mask);
    return result;
}
```

Let's take a closer look at how this works:

- The test `b != 0.0f` produces a `bool` result. We convert this into an unsigned integer by simply casting it. This results in either the value 1U (corresponding to `true`) or 0U (corresponding to `false`).
- We convert this unsigned result into a bit mask by subtracting it from 0U. Zero minus zero is still zero, and zero minus one is `-1`, which is `0xFFFFFFFFU` in 32-bit unsigned integer arithmetic.
- Next, we go ahead and calculate the quotient. We run this code *regardless* of the result of the non-zero test, thereby side-stepping any branch dependency issues.
- We now have our two answers ready to go: The quotient `q` and the default value `d`. We want to apply the mask in order to “select” one or the other value. But to do this, we need to *reinterpret* the floating-point bit patterns of `q` and `d` as if they were unsigned integers. The most portable way to accomplish this in C/C++ is to use a union containing

two members, one of which interprets a 32-bit value as a float, the other of which interprets it as an unsigned.

- The mask is applied as follows: We bitwise AND the quotient q with the mask, producing a bit pattern that matches q if the mask is all ones, but is all zeros if the mask is all zeros. We bitwise AND the default value d with the *complement* of the mask, yielding all zeros if the mask is all ones, or the bit pattern of d if the mask is all zeros. Finally, we bitwise OR these two values together, effectively selecting *either* the value of q or the value of d .

The use of a mask to select one of two possible values like this is called *predication* because we run both code paths (the one that returns a / b and the one that returns d) but each code path is *predicated* on the results of the test $(a \neq 0)$, via the mask. Because we are selecting one of two possible values, this is also often called a *select* operation.

Going to all this trouble to avoid a branch may seem like overkill. And it can be—its usefulness depends on the relative cost of a branch versus the predicated alternative on your target hardware. Predication really shines when a CPU’s ISAs provide special machine language instructions for performing a select operation. For example, the PowerPC ISA offers an integer select instruction `isel`, a floating-point select instruction `fsel`, and even a SIMD vector select instruction `vecsel`, and their use can definitely result in performance improvements on PowerPC based platforms (like the PS3).

It’s important to realize that predication only works when both branches can be executed *safely*. Performing a divide by zero operation in floating-point generates a quiet not-a-number (QNaN), but an integer divide by zero throws an exception that will crash your game (unless it is caught). That’s why we converted this example to floating-point before applying predication to it.

4.2.7 Superscalar CPUs

The pipelined CPU we described in Section 4.2.1 is what is called a *scalar* processor. This means that it can start executing at most one instruction per clock cycle. Yes, multiple instructions are “in flight” at any given moment, but only one *new* instruction is sent down the pipeline every clock cycle.

At its core, parallelism is about making use of multiple hardware components simultaneously. So one way to double the throughput of a CPU (at least in theory!) would be to *duplicate* most of the components on the chip, in such a way that *two* instructions could be launched each clock cycle. This is called a *superscalar* architecture.

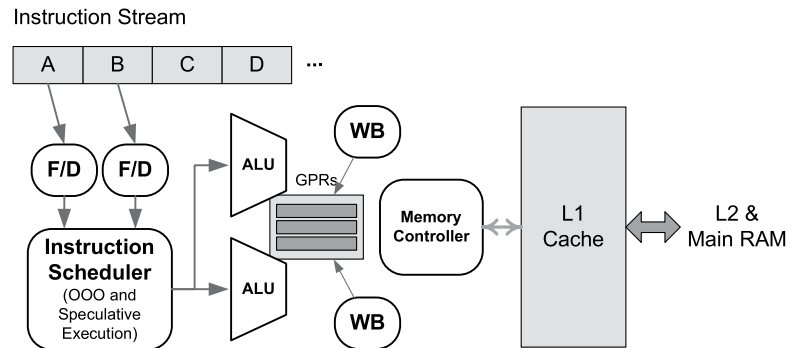


Figure 4.13. A pipelined superscalar CPU contains multiple execution components (ALUs, FPUs and/or VPUs) fed by a single instruction scheduler which typically supports out-of-order and speculative execution.

In a superscalar CPU, two (or more) instances of the circuitry that manages each stage of the pipeline¹ is present on-chip. The CPU still fetches instructions from a single instruction stream, but instead of issuing the one instruction pointed to by the IP during each clock cycle, the next *two* instructions are fetched and dispatched during each clock cycle. Figure 4.13 illustrates the hardware components found in a two-way superscalar CPU, and Figure 4.14 traces the path of ten instructions, “A” through “N,” as they move through this CPU’s two parallel pipelines.

4.2.7.1 Complexity of Superscalar Designs

Implementing a superscalar CPU isn’t quite as simple as “copying and pasting” two identical CPU cores onto a die. Although it’s reasonable to envision a superscalar CPU as two parallel instruction pipelines, these two pipelines are fed from a single instruction stream. Some kind of control logic is therefore required at the front end of these parallel pipes. Just as on a CPU that supports out-of-order execution, a superscalar CPU’s control logic looks ahead in the instruction stream in an attempt to identify *dependencies* between instructions, and then issues instructions out of order in an attempt to mitigate their effects.

In addition to data and branch dependencies, a superscalar CPU is prone to a third kind of dependency known as a *resource dependency*. This kind of dependency arises when two or more consecutive instructions all require the

¹Technically pipelining and superscalar designs are two independent forms of parallelism. A pipelined CPU needn’t be superscalar. Likewise, a superscalar CPU needn’t be pipelined, although the majority of them are.

Clock Cycle	F ₀	F ₁	D ₀	D ₁	E ₀	E ₁	M ₀	M ₁	W ₀	W ₁
0	A	B								
1	C	D	A	B						
2	E	F	C	D	A	B				
3	G	H	E	F	C	D	A	B		
4	I	J	G	H	E	F	C	D	A	B
5	K	L	I	J	G	H	E	F	C	D
6	M	N	K	L	I	J	G	H	E	F

Figure 4.14. Best-case execution of 14 instructions “A” through “N” on a superscalar pipelined CPU over seven clock cycles.

same functional unit within the CPU. For example, let’s imagine we have a superscalar CPU with two integer ALUs but only one FPU. Such a processor is capable of issuing two integer arithmetic instructions on each clock. But if two floating-point arithmetic instructions are encountered in the instruction stream, they cannot both be issued on the same clock cycle because the resource required by the second (the FPU) will already be in use by the first. As such, the control logic that manages instruction dispatch on a superscalar CPU is even more complex than that found on a scalar CPU that supports out-of-order execution.

4.2.7.2 Superscalar and RISC

A two-way superscalar CPU requires roughly two times the silicon real-estate of a comparable scalar CPU design. In order to free up transistors, most superscalar CPUs are therefore *reduced instruction set* (RISC) processors. The ISA of a RISC processor provides a comparatively small set of instructions, each with a very focused purpose. More complex operations are performed by building up sequences of these simpler instructions. In contrast, the ISA of a *complex instruction set computer* (CISC) offers a much wider variety of instructions, each of which may be capable of performing more complex operations.

4.2.8 Very Long Instruction Word (VLIW)

We saw in Section 4.2.7.1 that a superscalar CPU contains highly complex instruction dispatch logic. This logic takes up valuable real-estate on the CPU

die. Also, CPUs are only capable of looking ahead in the instruction stream by a relatively small number of instructions when analyzing dependencies and looking for opportunities for out-of-order and/or superscalar instruction dispatch. This limits the effectiveness of the dynamic optimizations a CPU can perform.

A somewhat simpler way to implement instruction level parallelism is to design a CPU that has multiple compute elements (ALUs, FPUs, VPU) on-chip, but leaves the task of *dispatching* instructions to those compute elements entirely to the programmer and/or compiler. That way, all the complex instruction-dispatch logic can be eliminated, and those transistors devoted instead to implementing more compute elements or a larger cache. As a side benefit, programmers and compilers ought to be better at optimizing the dispatching of the instructions in their programs than the CPU could ever be, because they can select instructions for dispatch from a much wider window (typically an entire function's worth of instructions).

To allow programmers and/or compilers to dispatch instructions to multiple compute elements on each clock cycle, the instruction word is extended so that it contains two or more "slots," each corresponding to a compute element on the chip. For example, if our hypothetical CPU contained two integer ALUs and two FPUs, a programmer or compiler would need to be able to encode up to two integer and two floating-point operations within each instruction word. We call this a *very long instruction word* (VLIW) design. The VLIW architecture is illustrated in Figure 4.15.

We can look to the Playstation 2 for a concrete example of VLIW architecture: The PS2 contained two coprocessors called *vector units* (VU0 and VU1), each of which was capable of dispatching two instructions per clock cycle. Each instruction word was comprised of two slots called *low* and *high*. It was often a challenge to fill both slots effectively when hand-coding in assembly language, although tools were developed that helped programmers to convert a one-instruction-per-clock program into an efficient two-instructions-per-clock format.

There are trade-offs between the superscalar and VLIW approaches. Because it lacks the complex scheduling, out-of-order execution and branch prediction logic of a superscalar CPU, a VLIW processor is much simpler, and can therefore potentially make heavier use of parallelism than its superscalar counterparts. However, it can be very tough to transform a serial program into a form that takes full advantage of the parallelism in a VLIW. This makes the job of the programmer and/or compiler more difficult. That said, a number of advances have been made to overcome some of these limitations, including variable-width VLIW designs. For example, see <http://researcher.watson>.

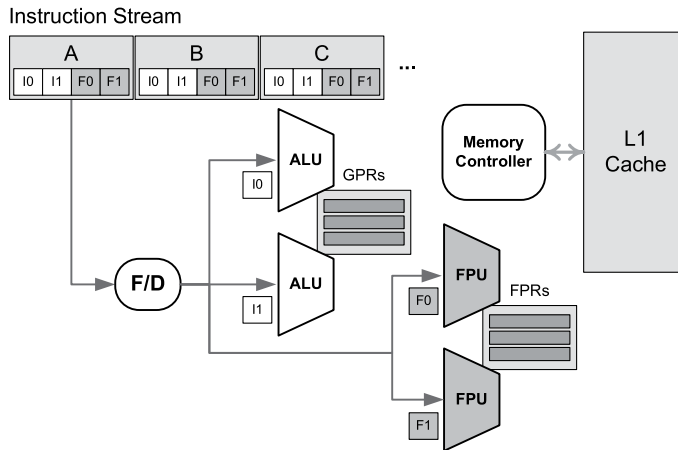


Figure 4.15. A pipelined VLIW CPU architecture consisting of two integer ALUs and two floating-point FPUs. Each very long instruction word consists of two integer and two floating-point operations, which are dispatched to the corresponding functional units. Notice the absence of the complex instruction scheduling logic that would be present in a superscalar CPU.

ibm.com/researcher/view_group_subpage.php?id=2834.

4.3 Explicit Parallelism

Explicit parallelism is designed to make concurrent software run more efficiently. Hence all explicitly parallel hardware designs permit more than one instruction stream to be processed in parallel. We'll list a few common explicitly parallel designs below, increasing in granularity from *hyperthreading* at the most fine-grained end of the spectrum to *cloud computing* at the most coarse-grained end.

4.3.1 Hyperthreading

As we saw in Section 4.2.5.2, some pipelined CPUs are capable of issuing instructions *out of order* as a means of reducing pipeline stalls. Normally a pipelined CPU executes instructions in program order; but sometimes the next instruction in the instruction stream cannot be issued due to a dependency on an in-flight instruction. This creates a *delay slot* into which another instruction could theoretically be issued. An OOO CPU can “look ahead” in the instruction stream and select an instruction to issue out-of-order into such a delay slot.

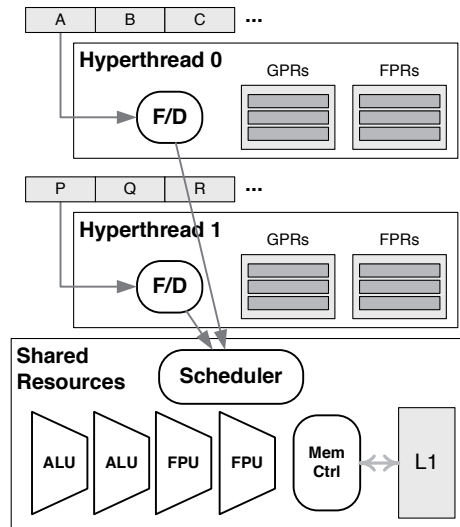


Figure 4.16. A hyperthreaded CPU containing two front ends (each consisting of a fetch/decode unit and a register file), but with a single back end containing ALUs, FPUs, a memory controller, an L1 cache, and an out-of-order instruction scheduler. The scheduler issues instructions from both front-end threads to the shared back-end components.

With only a single instruction stream, the CPU's options are somewhat limited when selecting an instruction to issue into a delay slot. But what if the CPU could select its instructions from two separate instruction streams at once? This is the principle behind a *hyperthreaded* (HT) CPU core.

Technically speaking, an HT core consists of two register files and two instruction decode units, but with a single "back end" for executing instructions, and a single shared L1 cache. This design enables an HT core to run two independent threads, while requiring fewer transistors than a dual core CPU, thanks to the shared back end and L1 cache. Of course, this sharing of hardware components also results in lower instruction throughput relative to a comparable dual core CPU, because the threads contend for these shared resources. Figure 4.16 illustrates the key components in a typical hyperthreaded CPU design.

4.3.2 Multicore CPUs

A CPU *core* can be defined as a self-contained unit capable of executing instructions from at least one instruction stream. Every CPU design we've looked at until now could therefore qualify as a "core." When more than one core is included on a single CPU die, we call it a *multicore* CPU.

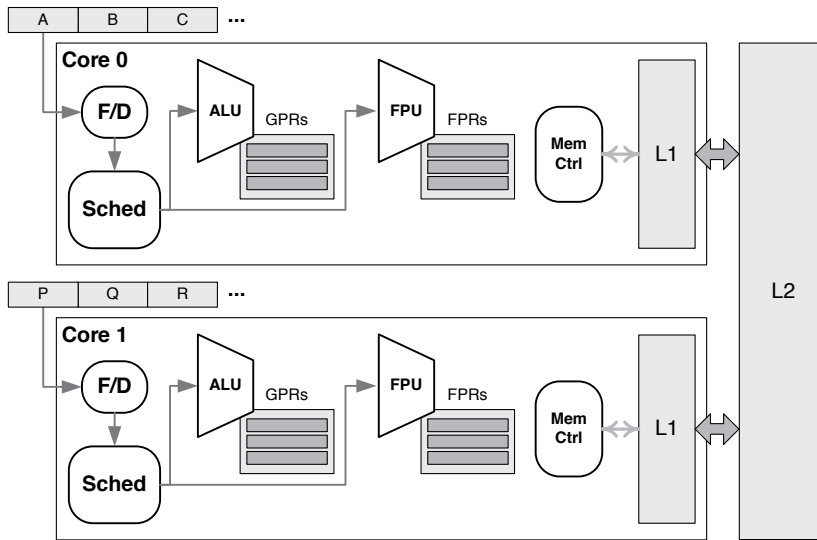


Figure 4.17. A simple multicore CPU design.

The specific design within each core can be any of the designs we’ve looked at thus far—each core might employ a simple serial design, a pipelined design, a superscalar architecture, a VLIW design, or might be a hyperthreaded core. Figure 4.17 illustrates a simple example of a multicore CPU design.

The PlayStation 4 and Xbox One game consoles both contain multicore CPUs. Each contains an *accelerated processing unit* (APU) consisting of two quad-core AMD Jaguar modules, integrated onto a single die with a GPU, memory controller and video codec. (Of these eight cores, seven are available for use by game applications. However, roughly half of the bandwidth on the seventh core is reserved for operating system use.) The Xbox One X also contains an eight-core APU, but its cores are based on proprietary technology developed in partnership with AMD, rather than on the Jaguar microarchitecture like its predecessor. Figure 4.18 shows a block diagram of the PS4 hardware architecture, and Figure 4.19 presents a block diagram of the Xbox One hardware architecture.

4.3.3 Symmetric versus Asymmetric Multiprocessing

The *symmetry* of a parallel computing platform has to do with how the CPU cores in the machine are treated by the operating system. In *symmetric multiprocessing* (SMP), the available CPU cores in the machine (provided by any combination of hyperthreading, multicore CPUs or multiple CPUs on a single

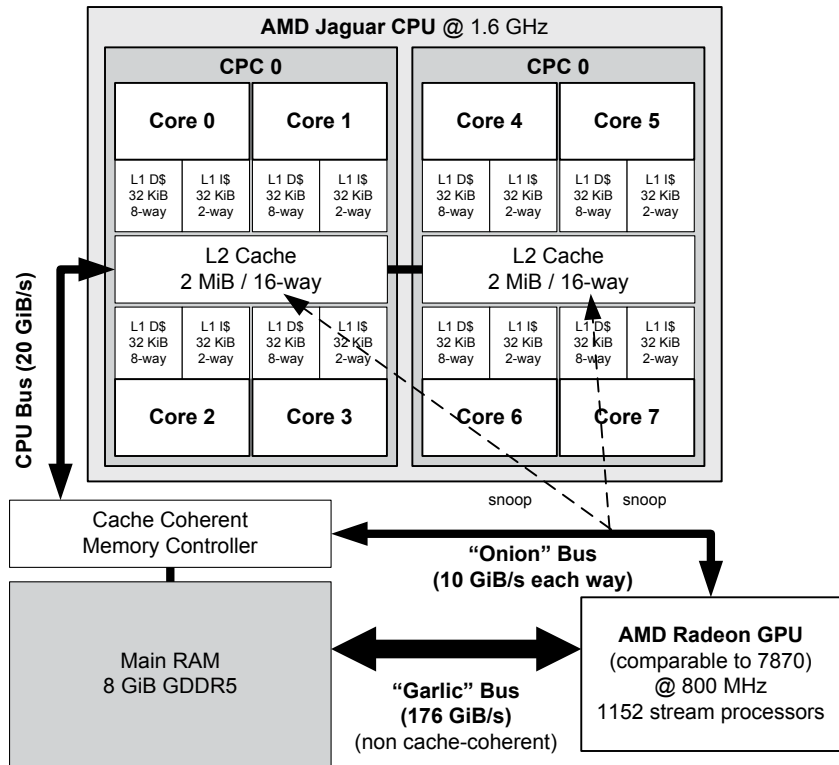


Figure 4.18. Simplified view of the PS4's architecture.

motherboard) are homogeneous in terms of design and ISA, and are treated equally by the operating system. Any thread can be scheduled to execute on any core. (Note, however, that it is possible in such systems to specify an *affinity* for a thread, causing it to be more likely, or even guaranteed, to be scheduled on a particular core.)

The PlayStation 4 and the Xbox One are examples of SMP. Both of these consoles contain eight cores, of which seven are available for use by the programmer, and the application is free to run threads on any of the available cores.

In *asymmetric multiprocessing* (AMP), the CPU cores are not necessarily homogeneous, and the operating system does not treat them equally. In AMP, one "master" CPU core typically runs the operating system, and the other cores are treated as "slaves" to which workloads are distributed by the master core.

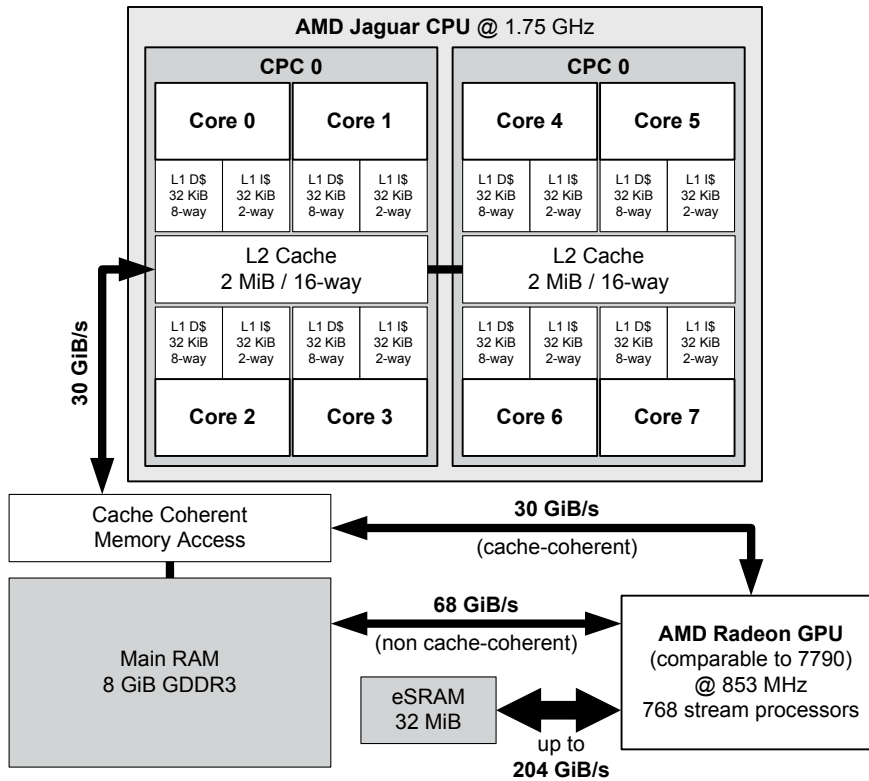


Figure 4.19. Simplified view of the Xbox One's architecture.

The *cell broadband engine* (CBE) used in the PlayStation 3 is an example of AMP; it employs a main CPU known as the “power processing unit” (PPU) which is based on the PowerPC ISA, along with eight coprocessors known as “synergistic processing units” (SPUs) which are based around a completely different ISA. (See Section 3.5.5 for more on the PS3’s hardware architecture.)

4.3.4 Distributed Computing

Yet another way to achieve parallelism in computing is to make use of multiple stand-alone computers working in concert. This is known as *distributed computing* in the most general sense. There are various ways to architect a distributed computing system, including:

- computer clusters,
- grid computing, and